

Lab 2: Fundamentals of Object-Oriented Programming – Encapsulation

Dr. Paweł Głaba

Season 2026

Basic Problem

Classes

Implement a class `Sandwich` with the following fields: **name**, **bread** (String or object of `Bread` class), and **ingredients**. The bread can be white or wholegrain and, in addition, it can be toasted or fresh. The ingredients can be of type meat, veggies, cheese, or sauce.

Store sandwich ingredients in a list. Each type of ingredient should have its own class with the following properties: weight in grams and a method for calculating its calories according to its type. Calories per gram are calculated using modifiers. Each ingredient has 2 calories per gram as the base and a modifier that gives the exact calories.

Calories

Bread modifiers:

- White – 1.4
- Wholegrain – 1.0
- Toasted – 1.1
- Fresh – 1.0

Ingredient modifiers:

- Meat – 1.3
- Veggies – 0.7
- Cheese – 1.2
- Sauce – 0.8

Example

Wholegrain toasted bread has modifiers 1.0 and 1.1, which means that 100 grams of such bread will have

$$(2 \text{ calories} \cdot 100) \cdot 1.0 \cdot 1.1 = 220.00$$

total calories.

Furthermore, cheese has a modifier of 1.2, so a cheese ingredient will have 2.4 calories per gram.

Data Validation

Create static factory methods in classes to validate input values. If the values are invalid, the method should return `null`.

- Check if the bread type is valid; otherwise, print the message "Invalid type of bread."
- Check if the bread preparation is valid; otherwise, print the message "Invalid bread preparation."
- Check if the bread weight is valid (range [1..200]); otherwise, print the message "Bread weight should be in the range [1..200]."
- Check if the ingredient weight is valid (range [1..50]); otherwise, print the message "[Ingredient type name] weight should be in the range [1..50]."
- Check if the name of the sandwich is empty or longer than 15 symbols; otherwise, print the message "Sandwich name should be between 1 and 15 symbols."

- Check if the number of ingredients is inside the range `[0..10]`; otherwise, print the message "The number of ingredients should be in range `[0..10]`."

Your task is to model the classes in such a way that they are properly encapsulated and provide a public method for every sandwich that calculates its calories according to the ingredients it has. Override the `toString()` method to show all sandwich ingredients.

Write a test program called `SandwichTester` and demonstrate the functionality of the `Sandwich` class.

Space Station Management

We will continue to expand the `Space Station Management` project. For the start, add the field `id` to the `CrewMember` class. Use the `UUID` class to generate a unique identifier.

Officer

Create a class called `Officer` that has the following properties:

- `id`,
- `name`,
- `rank`,
- `department`,
- `years of experience`.

Use `UUID` to generate a unique identifier.

Supply

Define a class `Supply` with properties such as:

- `name` (e.g. oxygen tank, food pack, medical kit),
- `quantity`,
- `cost` (use the `BigDecimal` class).

Module

For safety and organization reasons, crew members are assigned to different station modules. Create a `Module` class that contains an array of `CrewMember` objects with maximum capacity.

Implement methods to:

- add a crew member to a module,
- remove a crew member from a module using the crew member `id`.

SpaceStation

Design a class called `SpaceStation` that holds:

- a list of officers,
- an array of crew members,
- a list of supplies.

Implement methods for adding and removing crew members and officers (by `id`).

Our station is divided into various sections to support daily operations. Create a field – a two-dimensional array – that represents the station map, where each cell contains a `Module` object.

Remember, you need to hide anything that is not intended to be viewed or modified from outside the class.